

etho: A Python framework for coordinating stimuli, data acquisition, and hardware control in neuroscience experiments

Jan Clemens¹

¹ Carl von Ossietzky University, Oldenburg, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Frederick Boehm](#)

Reviewers:

- [@peircej](#)
- [@Grant-Giesbrecht](#)

Submitted: 15 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Etho is an open-source Python package for running behavioral and neuroscience experiments that require coordinated stimulus presentation, hardware control, and data acquisition. It was developed for experiments combining stimulus playback, analog and digital input/output, video acquisition, optogenetic stimulation, environmental monitoring, and external imaging systems. The package is user-friendly and does not require programming skills for configuration or operation. It provides text-based configuration through both a command-line interface and a graphical user interface for running experiments.

The central use case for etho is a laboratory experiment in which multiple devices must operate together with reproducible timing. For example, an experiment can present calibrated acoustic and optogenetic stimuli through National Instruments data-acquisition hardware, record multichannel microphone and trigger signals, acquire synchronized camera frames, and send control pulses to external systems such as ScanImage (Pologruto et al., 2003). After the experiment, the timing relationships between stimuli, recordings, and control signals remain available for downstream annotation and analysis. Experimental logic is specified in human-readable YAML protocol files and tabular stimulus playlists, separating scientific design from low-level device code and making experiments easier to inspect, modify, archive, and reproduce.

Statement of need

Behavioral neuroscience experiments are inherently multimodal. Experiments often combine controlled sensory stimulation, high-speed video, electrophysiology, optical imaging, optogenetics, and environmental monitoring. Commercial acquisition software and vendor-specific device tools can control individual hardware components, but are often poorly suited for experiments that require flexible coordination across heterogeneous systems.

Labs therefore often resort to custom scripts to integrate heterogeneous hardware environments. While such scripts can be flexible, they are often difficult to reuse across rigs, users, and projects, especially when experiments involve multiple asynchronous data streams and require precise logging of stimuli, timing, and metadata. These challenges are particularly acute in academic environments, where personnel turnover every 2–5 years can make long-term maintenance and reproducibility difficult. In addition, custom scripts often store metadata in idiosyncratic formats, limiting interoperability and complicating data reuse. Flexible, reproducible, and user-friendly experiment control software is therefore needed.

etho addresses this gap by providing a configurable Python framework for multi-device behavioral experiments. It is designed for laboratories that run repeated experimental protocols across one or more rigs and require the flexibility of a programmable system without embedding all

41 experimental logic directly in scripts. In contrast to vendor-specific solutions, the system is
 42 hardware-agnostic and extensible, allowing it to adapt as new hardware devices and experimental
 43 paradigms are introduced.

44 In *etho*, protocols define hardware configurations and acquisition settings, while stimulus
 45 playlists specify trial structure and stimulus parameters. This separation supports systematic
 46 experimental variation while keeping hardware settings, calibration information, and acquired
 47 data tightly linked and reproducibly associated with each experiment.

48 State of the field

49 Several open-source systems address parts of the experimental control problem, but they
 50 differ substantially in scope and design goals. Bonsai provides a powerful visual programming
 51 environment for asynchronous acquisition, online processing, and closed-loop control of data
 52 streams (Lopes et al., 2015). However, its workflow-centric design can make large experimental
 53 protocols difficult to standardize, version, and reuse across users and rigs, particularly when
 54 experiments require structured management of hardware configurations, stimulus playlists, and
 55 metadata. pyControl focuses on specifying behavioral tasks as microcontroller-based state
 56 machines together with tools for managing parallel setups (Akam et al., 2022), but is optimized
 57 for discrete trial logic rather than coordinated acquisition across heterogeneous devices such as
 58 cameras, audio interfaces, environmental sensors, and external imaging systems. Autopilot
 59 emphasizes distributed behavioral experiments using networked Raspberry Pis (Saunders et al.,
 60 2019), but is less suited for experiments requiring high-bandwidth synchronized acquisition
 61 or integration with specialized laboratory hardware. Bpod provides temporally precise control
 62 for trial-based rodent experiments through microcontroller-driven state machines (Solari et
 63 al., 2018), but does not aim to provide a general framework for multimodal acquisition and
 64 hardware orchestration. PsychoPy is widely used for visual and auditory stimulus presentation
 65 in psychophysics and human behavioral experiments (Peirce et al., 2019), but focuses primarily
 66 on stimulus delivery rather than coordinated control of diverse experimental hardware and
 67 multimodal data streams.

68 Ethocomplements these tools by focusing on reproducible coordination of heterogeneous
 69 laboratory hardware within a pragmatic Python framework. Rather than replacing existing
 70 acquisition or stimulus-generation software, it provides an architectural layer that organizes
 71 experiments around structured protocol, playlist, calibration, and log files. This design enables
 72 experiments to combine stimulus generation, analog and digital acquisition, camera recording,
 73 environmental monitoring, and external triggers while retaining direct Python-level control and
 74 support for lab-specific hardware extensions. Existing tools can therefore be integrated into
 75 ethoworkflows when appropriate, for example by using PsychoPy for calibrated visual stimulus
 76 generation. The primary contribution of *etho* is thus not a new standalone device-control
 77 system, but a flexible framework for coordinating multimodal experiments in a reproducible
 78 and extensible manner.

79 Software design

80 *etho* is organized modularly around **services**, **protocols**, **playlists**, and **callbacks** (Figure 1).
 81 Services define hardware interfaces. An experiment is defined by a protocol and a playlist.
 82 Protocols list and parameterize services and callbacks for processing and saving acquired data.
 83 Playlists specify trial structure and stimuli.

84 **Services** encapsulate hardware-specific functionality. Current service classes include support
 85 for multiple camera backends, National Instruments data-acquisition boards, sound playback,
 86 optogenetic LED control, environmental monitoring, DLP-projector stimulation, and remote
 87 control of ScanImage calcium imaging rigs.

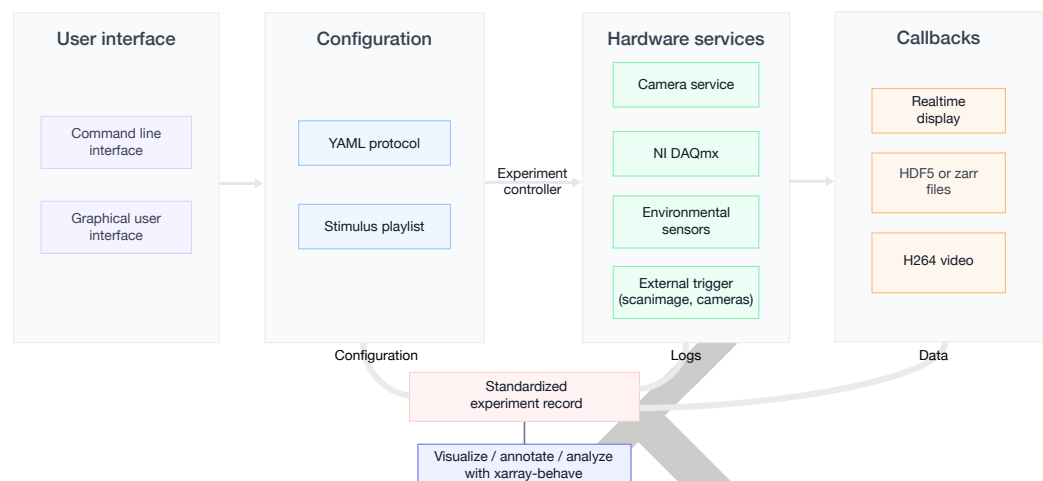


Figure 1: etho software structure and data flow: Command-line and graphical user interfaces allow users to select configuration files and run experiments. Protocol and playlist configuration files define and parameterize hardware and stimuli for each experiment. Hardware services control and log connected devices, while callbacks display and save data streams generated during experiments. Together, configuration files, logs, and acquired data form a standardized experimental record that can be further processed and analyzed.

88 **Protocols** are YAML files that define the structure of an experiment. A protocol specifies the
89 experiment duration, active services, service-specific configurations, and callbacks. For example,
90 a protocol can configure a camera service with frame rate, region of interest, exposure, and
91 video-saving callbacks, while also configuring a data-acquisition service with sampling rate,
92 analog input channels, analog and digital output channels, and callbacks for Zarr (Miles et
93 al., 2020) data storage. Because protocols are plain text, they can be version controlled and
94 stored alongside data and analysis code.

95 This modular service-plus-protocol structure makes etho easily extensible: New hardware can
96 be added as new Python classes implementing the service interface and registering a new
97 service. The new service can then be added to a protocol and used in an experiment.

98 **Playlists** are tabular files that define trial sequences. They can reference fixed, external waveform
99 files or simple runtime-generated stimuli, such as sinusoids, optogenetic LED pulse trains,
100 hardware clock signals, and hardware trigger pulses. Playlists support multichannel stimulation
101 with per-channel parameters. Speaker and LED calibration files convert requested stimulus
102 intensities into device-specific output amplitudes, helping preserve quantitative stimulus control
103 across rigs.

104 **Callbacks** process data produced by services and are specified in the protocols. Camera services
105 can display frames, save video, and save timestamps; data-acquisition services can save analog
106 data or plot traces. The callbacks also define standardized formats for data and metadata,
107 facilitating long-term data reuse.

108 In addition, etho writes separate logs for each service, recording service-specific events,
109 parameters, warnings, and errors during an experiment. These logs provide an audit trail
110 for diagnosing hardware or timing problems and help connect acquired data to the processes
111 that produced them. The callback and logging mechanisms separate data acquisition, online
112 monitoring, and experiment provenance from low-level hardware control code.

113 Services and callbacks run in independent processes, thus making use of modern multi-core
114 hardware. This concurrency is what makes etho fast, enabling precise synchronized stimulus
115 presentation and real-time, high-speed video compression using GPUs.

116 The modular service-process architecture makes the system reusable across rigs and labs and

maintainable in the long term. Explicit protocol and playlist files and standardized data formats encourage reproducibility, traceability, and reuse of experiments.

Research impact

Etho is currently used in the Roemschied, Deutsch, and Clemens laboratories to run behavioral and neuroscience experiments on acoustic communication, sensory and motor processing, and social behavior (Palacios-Muñoz & Clemens, 2025; Ravindran Nair et al., 2026; Steinfath et al., 2025; Vijendravarma et al., 2022). Its documentation includes installation instructions, protocol and playlist formats, GUI and command-line usage, and API references, making the software accessible and broadly usable.

The value of etholies in making complex experiments easier to run, maintain, and reproduce. By organizing experimental configuration, acquired data, calibration information, and log files into structured, versionable formats within a reusable Python package, ethoreduces dependence on one-off scripts and ad-hoc hardware integration. This organization enforces best practices, improves reproducibility, facilitates reuse across rigs and users, and supports more transparent reporting of experimental methods and acquisition parameters.

AI usage disclosure

During the preparation of this work, the author used ChatGPT to assist with language and phrasing throughout the manuscript. ChatGPT and codex were also used for drafting the documentation and for refactoring code. After using these services, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication and the software.

Acknowledgements

We thank past and current members of the Roemschied, Deutsch, and Clemens labs for testing etho on experimental rigs and reporting bugs. Development of etho was funded via an Emmy Noether Grant (Project number 329518246) and an ERC Starting Grant (Grant agreement No. 851210) to JC.

References

- Akam, T., Lustig, A., Rowland, J. M., Kapaniaiah, S. K., Esteve-Agraz, J., Panniello, M., Márquez, C., Kohl, M. M., Kätzel, D., Costa, R. M., & Walton, M. E. (2022). Open-source, Python-based, hardware and software for controlling behavioural neuroscience experiments. *eLife*, 11, e67846. <https://doi.org/10.7554/eLife.67846>
- Lopes, G., Bonacchi, N., Frazão, J., Neto, J. P., Atallah, B. V., Soares, S., Moreira, L., Matias, S., Itskov, P. M., Correia, P. A., Medina, R. E., Calcaterra, L., Dreosti, E., Paton, J. J., & Kampff, A. R. (2015). Bonsai: An event-based framework for processing and controlling data streams. *Frontiers in Neuroinformatics*, 9. <https://doi.org/10.3389/fninf.2015.00007>
- Miles, A., Kirkham, J., Durant, M., Bourbeau, J., Onalan, T., Hamman, J., Patel, Z., shikharsg, Rocklin, M., dussin, raphael, Schut, V., de Andrade, E. S., Abernathey, R., Noyes, C., sbalmer, bot, pyup io, Tran, T., Saalfeld, S., Swaney, J., ... Banihirwe, A. (2020). *Zarr-developers/zarr-python: V2.4.0*. <https://doi.org/10.5281/zenodo.3773450>
- Palacios-Muñoz, A., & Clemens, J. (2025). *Drosophila males integrate song and pheromones using context-specific strategies*. *Neuroscience*. <https://doi.org/10.1101/2025.07.25.666755>

- 159 Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman,
160 E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior*
161 *Research Methods*, 51(1), 195–203. <https://doi.org/10.3758/s13428-018-01193-y>
- 162 Pologruto, T. A., Sabatini, B. L., & Svoboda, K. (2003). ScanImage: Flexible software
163 for operating laser scanning microscopes. *BioMedical Engineering OnLine*, 2(1), 13.
164 <https://doi.org/10.1186/1475-925X-2-13>
- 165 Ravindran Nair, S., Palacios-Muñoz, A., Martineau, S., Nasr, M., & Clemens, J. (2026).
166 Sex-specific behavioral feedback modulates sensorimotor processing and drives flexible
167 social behavior. *Nature Communications*, 17(1), 4026. <https://doi.org/10.1038/s41467-026-72057-9>
168
- 169 Saunders, J. L., Ott, L. A., & Wehr, M. (2019). *AUTOPILOT*: Automating Experiments with
170 Lots of Raspberry Pis. *Animal Behavior and Cognition*. <https://doi.org/10.1101/807693>
- 171 Solari, N., Sviatkó, K., Laszlovszky, T., Hegedüs, P., & Hangya, B. (2018). Open Source Tools
172 for Temporally Controlled Rodent Behavior Suitable for Electrophysiology and Optogenetic
173 Manipulations. *Frontiers in Systems Neuroscience*, 12, 18. <https://doi.org/10.3389/fnsys.2018.00018>
174
- 175 Steinfath, E., Khalili, A., Stenger, M., Schultze, B. L., Nair, S. R., Alizadeh, K., & Clemens, J.
176 (2025). A neural circuit for context-dependent multimodal signaling in *Drosophila*. *Nature*
177 *Communications*, 16(1), 9472. <https://doi.org/10.1038/s41467-025-64907-9>
- 178 Vijendravarma, R. K., Narasimha, S., Steinfath, E., Clemens, J., & Leopold, P. (2022).
179 *Drosophila* females have an acoustic preference for symmetric males. *Proceedings of the*
180 *National Academy of Sciences*, 119(13), e2116136119. <https://doi.org/10.1073/pnas.2116136119>
181